2교시: Pipeline 심화 — 다양한 태스크와 한국어 모델 활용

> **학습 목표**: Pipeline을 활용하여 다양한 NLP 태스크(감성 분석, 요약, 번역, 제로샷 분류)를 수행하고, 한국어에 적합한 모델을 선택하여 실습한다. Pipeline 내부 동작(Tokenizer, Model)의 개념을 이해한다.

> **실습 환경: Google Colab (TPU)**
>
> 이번 교시에서는 여러 모델을 다운로드하고 실행해야 하므로, 로컬 PC 대신 **Google Colab**을 사용합니다. Colab은 구글이 제공하는 무료 클라우드 실습 환경으로, 브라우저만 있으면 **TPU/GPU를 무료로** 사용할 수 있습니다.
>
> **Colab 실습 준비 (3단계)**:
>
> 1. colab.research.google.com 접속 (구글 계정 필요)
> 2. **새 노트** 클릭
> 3. 메뉴 **런타임** → **런타임 유형 변경** → **TPU** 선택 후 저장
> 4. 첫 번째 셀에 아래 명령어 실행
>
> ```
> !pip install -q transformers==4.51.3 torch==2.6.0
> ```
>
> - Colab에는 `transformers`, `torch`가 기본 설치되어 있지만, 위 명령어로 최신 버전을 보장합니다.
> - 무료 티어 기준 사용 시간에 제한이 있으므로, 사용하지 않을 때는 **런타임** → **런타임 연결 해제**를 해주세요.
> - 코드에서 `device` 설정은 **자동 감지 패턴**을 사용합니다. GPU가 있으면 GPU를, 없으면 CPU를 자동으로 선택하므로 **환경에 관계없이 동일한 코드**로 실행됩니다.
>
> ```
> # GPU/CPU 자동 감지 — 환경에 관계없이 동일 코드 사용
> import torch
> device = 0 if torch.cuda.is_available() else -1
> classifier = pipeline("sentiment-analysis", device=device)
> ```

---

1. Pipeline 심화 — 다양한 태스크 활용

도입 — 1교시 연결

> "1교시에서는 `pipeline("sentiment-analysis")`로 감성 분석 **한 가지**만 해봤습니다. 그런데 HuggingFace Pipeline은 감성 분석 말고도 **수십 가지 태스크**를 지원합니다. 모델만 바꾸면 같은 코드 구조로 번역도 하고, 요약도 하고, 분류도 할 수 있어요."

Pipeline이 지원하는 주요 태스크 목록:

태스크 이름	설명	예시

태스크 이름	설명	예시
`sentiment-analysis`	감성 분석 (긍정/부정)	"I love this!" → POSITIVE
`text-classification`	텍스트 분류 (범용)	스팸 분류, 주제 분류 등
`zero-shot-classification`	제로샷 분류 (카테고리 자유 지정)	뉴스 카테고리 분류
`summarization`	텍스트 요약	긴 기사 → 핵심 요약
`translation_xx_to_yy`	번역	영어 → 프랑스어
`text-generation`	텍스트 생성	문장 이어 쓰기
`question-answering`	질의응답	지문 + 질문 → 답변
`fill-mask`	빈칸 채우기	"I love [MASK]" → "I love you"
`ner`	개체명 인식	"서울에서 삼성전자" → 장소, 조직

> 이 중에서 오늘은 **텍스트 분류, 요약, 제로샷 분류, 번역** 4가지를 직접 실습해봅니다.

1-1. 텍스트 분류 — 특정 모델 지정하기

1교시에서는 `pipeline("sentiment-analysis")`로 **기본 모델**을 사용했다. 이번에는 **모델을 직접 지정**해본다.

```
import torch
from transformers import pipeline

device = 0 if torch.cuda.is_available() else -1

# 모델을 직접 지정하여 파이프라인 생성
classifier = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english",
    device=device
)

result = classifier("I love programming!")
print(result)
# [{'label': 'POSITIVE', 'score': 0.9998}]
```

코드 해설:

```
pipeline(
    "text-classification",          ← 태스크 이름 (어떤 종류의 작업을 할 것인지)
    model="distilbert-base-..."     ← HuggingFace Hub의 모델 ID (어떤 모델을 쓸 것인지)
)
```

모델 지정이 중요한 이유:

```
[model 파라미터 생략 시]
pipeline("sentiment-analysis")
→ HuggingFace가 "이 태스크에 적합한 기본 모델"을 자동 선택
→ 라이브러리 버전에 따라 기본 모델이 바뀔 수 있음

[model 파라미터 지정 시]
pipeline("text-classification", model="distilbert-base-uncased-finetuned-sst-2-
english")
→ 항상 같은 모델이 사용됨 (재현 가능)
→ 실무에서는 반드시 모델을 지정하는 것이 좋은 습관!
```

> DistilBERT는 큰 모델의 지식을 작은 모델로 증류(distil)한 NLP 모델입니다.

모델 ID 찾는 방법:

1. huggingface.co 접속
2. 상단 검색창에서 태스크 또는 키워드 검색 (예: "sentiment analysis")
3. 원하는 모델 클릭
4. 모델 페이지 상단의 **모델 이름**이 곧 **모델 ID**

```
브라우저 URL:  https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-
english
                                        └──────────── 이 부분이 모델 ID ────────────┘

코드에서 사용:  pipeline("text-classification", model="distilbert-base-uncased-
finetuned-sst-2-english")
```

> 태스크에 맞는 모델을 써야 합니다. `text-classification` 태스크에 번역 모델을 넣으면 에러가 납니다. 모델 카드에서 해당 모델이 어떤 태스크용인지 확인하세요.

---

2. 한국어 Pipeline 실습

2-1. 한국어 감성 분석

> **핵심**: 1교시에서 한국어가 잘 안 되는 이유는 모델이 **영어 전용**이었기 때문이다. **한국어 모델**을 지정하면 해결된다!

```
[1교시에서 한 것]
pipeline("sentiment-analysis")  ← 기본 모델: 영어 전용 (distilbert-base-
uncased-...)

"이 영화 재미있어요!" 입력
→ 모델은 영어만 학습했음 → 엉뚱한 결과 출력 😵
```

```
[해결: 다국어(multilingual) 모델 사용]
pipeline("sentiment-analysis", model="nlptown/bert-base-multilingual-uncased-
sentiment")

"이 영화 재미있어요!" 입력
→ 이 모델은 6개 언어(영/독/불/스페인/이탈리아/네덜란드)로 학습됨
→ 한국어는 학습 데이터에 포함되지 않아 결과가 불안정할 수 있음 ⚠
→ 참고용으로만 사용하고, 실무에서는 한국어 전용 모델을 사용하세요
```

```python
import torch
from transformers import pipeline

device = 0 if torch.cuda.is_available() else -1

# 한국어 감성 분석 모델 (다국어 지원)
ko_classifier = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment",
    device=device
)

# 한국어 테스트
results = ko_classifier([
    "이 영화 정말 재미있었어요!",
    "음식이 너무 맛없었습니다.",
    "그냥 보통이었어요."
])

for r in results:
    stars = int(r['label'][0])  # "5 stars" → 5
    print(f"{'★' * stars} ({r['label']}, 확신도: {r['score']:.4f})")
```

예상 출력:

```
★★★★★ (5 stars, 확신도: 0.6847)
★ (1 star, 확신도: 0.7234)
★★★ (3 stars, 확신도: 0.4512)
```

2-2. 한국어에 적합한 모델 목록

태스크	모델 ID	설명	한국어 성능
감성 분석	`nlptown/bert-base-multilingual-uncased-sentiment`	다국어 별점 분류 (1~5)	★★★ 보통
감성 분석	`snunlp/KR-FinBert-SC`	한국어 금융 감성 분류	★★★★★ 우수

태스크	모델 ID	설명	한국어 성능
감성 분석	`matthewburke/korean_sentiment`	한국어 긍정/부정	★★★★ 좋음
텍스트 분류	`klue/bert-base`	한국어 BERT (범용)	★★★★★ 우수
제로샷 분류	`joeddav/xlm-roberta-large-xnli`	다국어 제로샷 (한국어 비공식 지원)	★★★ 보통
요약	`digit82/kobart-summarization`	한국어 텍스트 요약	★★★★★ 우수
번역 (한→영)	`Helsinki-NLP/opus-mt-ko-en`	한국어 → 영어	★★★★ 좋음
번역 (영→한)	`Helsinki-NLP/opus-mt-en-ko`	영어 → 한국어	★★★★ 좋음
텍스트 생성	`skt/ko-gpt-trinity-1.2B-v0.5`	한국어 GPT (SK텔레콤)	★★★★ 좋음

> HuggingFace Hub에서 "korean"으로 검색하면 더 많은 한국어 모델을 찾을 수 있습니다.

2-3. 한국어 감성 분석 — 전용 모델 비교

> 📁 소스 파일: `source/5_week/ko_sentiment_compare.py`

```
import torch
from transformers import pipeline

device = 0 if torch.cuda.is_available() else -1

# ── 모델 1: 다국어 모델 (별점 1~5) ──
multi_clf = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment",
    device=device
)

# ── 모델 2: 한국어 감성 분석 전용 ──
ko_clf = pipeline(
    "sentiment-analysis",
    model="matthewburke/korean_sentiment",
    device=device
)

texts = [
    "이 제품 정말 좋아요! 강력 추천합니다.",
    "배송이 너무 늦고 포장도 엉망이었어요.",
    "가격은 괜찮은데 품질이 아쉬워요.",
]
```

```
print("=" * 60)
print("다국어 모델 vs 한국어 전용 모델 비교")
print("=" * 60)

for text in texts:
    r1 = multi_clf(text)[0]
    r2 = ko_clf(text)[0]
    print(f"\n입력: \"{text}\"")
    print(f"  다국어 모델: {r1['label']:>10s} ({r1['score']:.4f})")
    print(f"  한국어 전용: {r2['label']:>10s} ({r2['score']:.4f})")
```

예상 출력:

```
============================================================
다국어 모델 vs 한국어 전용 모델 비교
============================================================

입력: "이 제품 정말 좋아요! 강력 추천합니다."
  다국어 모델:    5 stars (0.7265)
  한국어 전용:    LABEL_1 (0.9728)

입력: "배송이 너무 늦고 포장도 엉망이었어요."
  다국어 모델:    2 stars (0.4615)
  한국어 전용:    LABEL_0 (0.9591)

입력: "가격은 괜찮은데 품질이 아쉬워요."
  다국어 모델:    2 stars (0.4764)
  한국어 전용:    LABEL_0 (0.9328)
```

> - 다국어 모델은 **별점(1~5 stars)**으로, 한국어 전용 모델은 **LABEL_0(부정)/LABEL_1(긍정)**으로 결과를 출력합니다.
> - 한국어 전용 모델이 확신도(score)가 더 높고, 긍정/부정 판단이 더 명확합니다.

> 한국어 전용 모델이 더 정확한 경우가 많습니다. 프로젝트에서 한국어 데이터를 다룰 때는 한국어 전용 모델을 우선 검토하세요.

2-4. 한국어 번역

> 🗀 소스 파일: `source/5_week/ko_translate.py`

```
import torch
from transformers import MarianMTModel, MarianTokenizer

def translate(text, model_name):
    device = "cuda" if torch.cuda.is_available() else "cpu"
    tokenizer = MarianTokenizer.from_pretrained(model_name)
    model = MarianMTModel.from_pretrained(model_name).to(device)
    inputs = tokenizer([text], return_tensors="pt", padding=True).to(device)
```

```
    translated = model.generate(**inputs)
    return tokenizer.decode(translated[0], skip_special_tokens=True)

# 한→영: opus-mt-ko-en (존재 O)
print("한→영:", translate("오늘 날씨가 정말 좋습니다.", "Helsinki-NLP/opus-mt-ko-en"))

# 영→한: opus-mt-tc-big-en-ko (존재 O) ← 이걸 써야 함
print("영→한:", translate("I love learning artificial intelligence!", "Helsinki-NLP/opus-mt-tc-big-en-ko"))
```

예상 출력:

```
한→영: It's a great day.
영→한: US 92년 전
```

> ⚠ 한→영 번역은 자연스럽지만, 영→한 번역은 정확하지 않습니다. 무료 경량 번역 모델의 한계이며, 높은 품질의 한국어 번역이 필요하다면 GPT, Papago 등의 API를 사용하는 것이 좋습니다.

MarianMT란?

> MarianMT는 **번역 전용** 경량 모델로, Helsinki 대학의 OPUS 프로젝트에서 만들었습니다. 언어 쌍별로 전용 모델이 존재하며 (예: `opus-mt-ko-en` = 한→영), 빠르고 가벼워서 로컬 환경에서도 부담 없이 사용할 수 있습니다.

코드 해설:

```
def translate(text, model_name):
    device = "cuda" if torch.cuda.is_available() else "cpu"   ← GPU 있으면 GPU, 없으면 CPU 자동 선택
    tokenizer = MarianTokenizer.from_pretrained(model_name)    ← 텍스트를 토큰(숫자)으로 변환하는 도구 로드
    model = MarianMTModel.from_pretrained(model_name).to(device) ← 번역 모델 로드 후 GPU/CPU에 배치
    inputs = tokenizer([text], return_tensors="pt", padding=True).to(device) ← 입력 텍스트를 모델이 이해하는 텐서로 변환
    translated = model.generate(**inputs)                      ← 번역 실행 (토큰 ID 시퀀스 생성)
    return tokenizer.decode(translated[0], skip_special_tokens=True) ← 토큰 ID를 다시 사람이 읽을 수 있는 텍스트로 변환
```

> 이 코드는 Pipeline 없이 **Tokenizer + Model을 직접 사용**하는 방식입니다. Pipeline이 내부에서 자동으로 해주던 3단계(토큰화 → 모델 추론 → 후처리)를 직접 작성한 것입니다.

2-5. 한국어 텍스트 요약

이 코드는 실행되지 않습니다.

```python
import torch
from transformers import pipeline

device = 0 if torch.cuda.is_available() else -1

# 한국어 요약 모델 (KoBART)
summarizer = pipeline("summarization", model="digit82/kobart-summarization",
device=device)

article = """
인공지능(AI) 기술이 다양한 산업에 빠르게 확산되고 있다. 의료 분야에서는 AI를 활용한
질병 진단 시스템이 의사의 판단을 보조하고 있으며, 금융 분야에서는 이상 거래 탐지와
신용 평가에 AI가 도입되었다. 교육 분야에서도 개인 맞춤형 학습 시스템이 등장하여
학생들의 학습 효율을 높이고 있다. 전문가들은 앞으로 AI가 더 많은 직업에 영향을
미칠 것으로 전망하면서도, 윤리적 문제와 일자리 변화에 대한 사회적 논의가 필요하다고
강조하고 있다.
"""

result = summarizer(article, max_length=80, min_length=20)
print("원문 길이:", len(article), "자")
print("요약 결과:", result[0]['summary_text'])
```

> 기본 영어 요약 모델과 달리 `digit82/kobart-summarization`은 한국어를 직접 요약할 수 있습니다.

2-6. Pipeline이 안 되는 한국어 모델 — 직접 사용하기

> **한국어 모델은 Pipeline과 호환되지 않는 경우가 많습니다.** 영어권 모델은 대부분 Pipeline 규격에 맞춰 config가 설정되어 있지만, 한국어 모델은 개인/연구실에서 만든 경우가 많아 config 설정이 누락되거나 Pipeline 규격과 맞지 않는 경우가 자주 있습니다.

```
[Pipeline 호환 모델]
pipeline("summarization", model="digit82/kobart-summarization")
→ ☑ 2~3줄로 바로 사용 가능

[Pipeline 비호환 모델]
pipeline("summarization", model="gogamza/kobart-summarization")
→ ✕ config 충돌 에러 발생
→ 해결: Tokenizer + Model을 직접 로드하여 사용
```

Pipeline 비호환 모델 사용 예시 — 한국어 요약 (gogamza/kobart-summarization):

> 🗀 소스 파일: `source/5_week/ko_summarize.py`

```python
import torch
from transformers import PreTrainedTokenizerFast, BartForConditionalGeneration

# ── 1단계: Tokenizer + Model 직접 로드 ──
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
tokenizer = PreTrainedTokenizerFast.from_pretrained("gogamza/kobart-summarization")
model = BartForConditionalGeneration.from_pretrained("gogamza/kobart-summarization").to(device)

article = """
인공지능(AI) 기술이 다양한 산업에 빠르게 확산되고 있다. 의료 분야에서는 AI를 활용한 질병 진단 시스템이 의사의 판단을 보조하고 있으며, 금융 분야에서는 이상 거래 탐지와 신용 평가에 AI가 도입되었다. 교육 분야에서도 개인 맞춤형 학습 시스템이 등장하여 학생들의 학습 효율을 높이고 있다. 전문가들은 앞으로 AI가 더 많은 직업에 영향을 미칠 것으로 전망하면서도, 윤리적 문제와 일자리 변화에 대한 사회적 논의가 필요하다고 강조하고 있다.
"""

# ── 2단계: 토큰화 (텍스트 → 숫자) ──
inputs = tokenizer([article], return_tensors="pt", max_length=1024, truncation=True).to(device)

# ── 3단계: 모델 추론 (요약 생성) ──
summary_ids = model.generate(inputs["input_ids"], max_length=80, min_length=20, num_beams=4)

# ── 4단계: 후처리 (숫자 → 텍스트) ──
result = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
print("요약 결과:", result)
```

코드 해설:

```
Pipeline이 자동으로 해주던 3단계를 직접 작성한 것입니다:

[Pipeline 방식]                    [직접 사용 방식]
pipeline("summarization",         tokenizer = ...from_pretrained(모델명)  ← Tokenizer 로드
  model="모델명")                  model = ...from_pretrained(모델명)      ← Model 로드
                                   inputs = tokenizer(텍스트)              ← 1단계: 토큰화
result = summarizer(텍스트)        summary_ids = model.generate(inputs)    ← 2단계: 추론
                                   result = tokenizer.decode(summary_ids)  ← 3단계: 후처리
```

> **정리**: Pipeline이 작동하면 Pipeline을 쓰고, 에러가 나면 모델 카드의 예시 코드를 참고하여 Tokenizer + Model을 직접 로드하세요. 한국어 모델을 사용할 때는 이 패턴을 알아두면 대부분의 모델을 사용할 수 있습니다.

---

3. 텍스트 요약과 제로샷 분류

3-1. 텍스트 요약 (영어)

```python
import torch
from transformers import pipeline

device = 0 if torch.cuda.is_available() else -1

summarizer = pipeline("summarization", device=device)

article = """
Artificial intelligence has transformed many industries in recent years.
From healthcare to finance, AI systems are being deployed to automate
tasks that previously required human intelligence. Machine learning,
a subset of AI, has been particularly impactful, enabling computers
to learn from data without being explicitly programmed. Deep learning,
which uses neural networks with many layers, has achieved remarkable
results in image recognition, natural language processing, and game playing.
However, concerns about AI safety, bias, and job displacement continue
to be important topics of discussion in both academic and public spheres.
"""

result = summarizer(article, max_length=50, min_length=20)
print("원문 길이:", len(article))
print("요약 결과:", result[0]['summary_text'])
```

요약 파라미터:

파라미터	의미	설명
`max_length`	요약문의 최대 토큰 수	줄이면 더 짧은 요약
`min_length`	요약문의 최소 토큰 수	늘리면 더 자세한 요약

> 요약 모델은 입력 텍스트가 너무 짧으면 (30단어 미만) 제대로 작동하지 않을 수 있습니다. 최소 50단어 이상의 텍스트를 넣는 것이 좋습니다.

3-2. 제로샷 분류 (Zero-Shot Classification)

> 일반적인 분류 모델은 '긍정/부정'처럼 **정해진 카테고리**만 분류할 수 있습니다. **제로샷 분류**는 카테고리를 자유롭게 지정할 수 있습니다.

```
[일반 분류 모델]
학습 단계: "기술" 1000개 + "스포츠" 1000개로 훈련 → 훈련 데이터 필요!
사용 단계: 새 텍스트 → "기술" or "스포츠" → 훈련한 카테고리만 분류 가능

[제로샷 분류 모델]
학습 단계: "의미 이해 능력"을 학습 → 특정 카테고리 학습 불필요!
사용 단계: 새 텍스트 + 자유로운 카테고리 → 아무 카테고리나 지정 가능!

"Zero-Shot" = "0번 학습(shot)으로도 분류 가능"
```

전통적인 분류 vs 제로샷 분류

구분	전통적인 분류 (Supervised)	제로샷 분류 (Zero-Shot)
사전 학습	각 카테고리별 대량의 데이터 필요	특정 카테고리 학습이 필요 없음
새로운 클래스 추가	모델을 다시 학습시켜야 함	라벨 이름만 추가하면 즉시 가능
유연성	정해진 것만 분류 가능 (폐쇄적)	어떤 주제든 분류 가능 (개방적)
정확도	특정 도메인에서 매우 높음	특정 도메인에서는 다소 낮을 수 있음

```
import torch
from transformers import pipeline

device = 0 if torch.cuda.is_available() else -1

zero_classifier = pipeline("zero-shot-classification", device=device)

text = "The new iPhone 16 has an amazing camera with AI features."

# 분류할 카테고리를 자유롭게 지정
result = zero_classifier(
    text,
    candidate_labels=["technology", "sports", "politics", "entertainment"]
)

print(f"텍스트: {text}\n")
for label, score in zip(result['labels'], result['scores']):
    bar = "▇" * int(score * 30)
    print(f"  {label:15s} {score:.4f} {bar}")
```

카테고리를 바꿔서 실험해보기:

```
# 실험 1: 감정으로 분류
result = zero_classifier(
    "i got a gift!",
    candidate_labels=["happy", "sad", "angry", "surprised"]
)

# 실험 2: 한국어 텍스트 + 영어 카테고리 (더 정확!)
result = zero_classifier(
    "삼성전자가 새로운 AI 칩을 발표했습니다.",
    candidate_labels=["technology", "sports", "politics", "entertainment"]
)
```

> **팁**: 한국어 텍스트를 제로샷 분류할 때는 카테고리를 **영어로** 지정하면 정확도가 올라갑니다.

---

4. Pipeline 내부 동작 — 개념 요약

> Pipeline 내부의 3단계(Tokenizer → Model → 후처리)는 1교시에서 이미 다뤘습니다. 여기서는 **Pipeline과 직접 사용 방식의 차이**만 정리합니다.

비교 항목	Pipeline	Model + Tokenizer 직접 사용
코드량	2~3줄	10줄 이상
난이도	쉬움	중간
유연성	제한적	높음 (중간 출력 확인, 커스텀 가능)
사용 시점	빠른 프로토타입, 데모, 프로젝트	파인튜닝, 커스텀 전/후처리

> **이 수업에서는 Pipeline 방식을 기본으로 사용합니다.** 다만 한국어 모델의 경우 Pipeline이 지원되지 않는 모델이 더 많으므로, 지원되지 않는 경우에는 Model+Tokenizer를 직접 사용합니다.

---

5. GPU 활용법

Pipeline에서 GPU 사용하기

```
import torch

# GPU/CPU 자동 감지 패턴 (권장)
device = 0 if torch.cuda.is_available() else -1
classifier = pipeline("sentiment-analysis", device=device)

# GPU가 있으면 device=0 (GPU), 없으면 device=-1 (CPU)이 자동 선택됩니다.
```

> `torch.cuda.is_available()`로 GPU 유무를 자동 감지하므로, GPU가 없는 환경에서도 에러 없이 CPU로 실행됩니다. GPU 유무를 수동으로 확인하려면 `nvidia-smi` 명령어를 실행하세요.

GPU 메모리 관리

여러 모델을 연속으로 로드하면 메모리가 부족해질 수 있습니다.

> `torch.cuda.empty_cache()`는 PyTorch가 내부적으로 캐시해둔 **미사용 메모리**만 정리합니다. 실제로 모델이 점유한 메모리를 해제하려면 **모델 변수를 `del`로 먼저 삭제**해야 합니다.

```
import torch, gc

def clear_gpu_memory():
    """GPU 메모리를 정리합니다."""
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    print("✅ GPU 메모리 정리 완료!")

# 모델 교체 전에 실행하는 올바른 순서:
```

```
del classifier          # 1. 모델 변수 삭제 (메모리 해제 대상으로 표시)
clear_gpu_memory()      # 2. 가비지 컬렉션 + 캐시 정리
```

CPU 환경 참고

GPU가 없는 환경에서도 실습은 가능합니다.

모델	GPU	CPU
distilbert (감성 분석)	~0.1초	~1초
bert-base (분류)	~0.2초	~3초
bart-large (요약/제로샷)	~1초	~10초

> CPU에서도 `distilbert` 등 경량 모델은 1~3초 내로 동작하므로 실습에 지장은 없습니다.

---

6. 실습: 한국어 Pipeline 종합 체험

실습 과제

> 🗀 소스 파일: `source/5_week/pipeline_tasks.py`
>
> VSCode에서 `5_week/pipeline_tasks.py`를 만들고, 아래 코드를 작성하여 실행해보세요.

```
"""
5주차 실습 2: 다양한 Pipeline 태스크 체험
"""

import torch
from transformers import pipeline

device = 0 if torch.cuda.is_available() else -1

# ── 1. 감성 분석 (한국어 지원) ──
print("=" * 60)
print("1. 다국어 감성 분석")
print("=" * 60)

classifier = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment",
    device=device
)

texts = [
    "이 영화 정말 재미있었어요!",
    "서비스가 너무 불친절했습니다.",
    "가격 대비 괜찮았어요."
]
```

```
for text in texts:
    result = classifier(text)[0]
    stars = int(result['label'][0])
    print(f"  {'★' * stars} {result['label']:>10s} | \"{text}\"")

# ── 2. 제로샷 분류 ──
print(f"\n{'=' * 60}")
print("2. 제로샷 분류")
print("=" * 60)

zero_clf = pipeline("zero-shot-classification", device=device)

news = "삼성전자가 새로운 AI 칩을 발표했습니다."
categories = ["technology", "sports", "politics", "entertainment"]

result = zero_clf(news, candidate_labels=categories)
print(f"  뉴스: \"{news}\"")
for label, score in zip(result['labels'], result['scores']):
    bar = "▇" * int(score * 30)
    print(f"    {label:>15s}: {score:.4f} {bar}")

# ── 3. 한국어 → 영어 번역 ──
print(f"\n{'=' * 60}")
print("3. 한국어 → 영어 번역")
print("=" * 60)

translator = pipeline("translation", model="Helsinki-NLP/opus-mt-ko-en",
device=device)

ko_texts = [
    "오늘 날씨가 정말 좋습니다.",
    "인공지능 수업이 재미있어요.",
    "빅데이터 분석 프로젝트를 진행하고 있습니다."
]

for text in ko_texts:
    result = translator(text)
    print(f"  한국어: {text}")
    print(f"  영  어: {result[0]['translation_text']}\n")

print("=" * 60)
print("모든 태스크 완료!")
print("=" * 60)
```

실습 중 자주 발생하는 문제와 해결

문제	원인	해결
`ModuleNotFoundError: No module named 'transformers'`	패키지 미설치	`pip install transformers==4.51.3`

문제	원인	해결
`ModuleNotFoundError: No module named 'torch'`	PyTorch 미설치	`pip install torch==2.6.0`
첫 실행이 매우 느림 (1~5분)	모델 다운로드 중	정상! 두 번째부터는 캐시 사용
`OSError: Connection error`	인터넷 연결 문제	와이파이 확인, 재시도
`device=0`인데 에러	GPU 미지원 환경	`device = 0 if torch.cuda.is_available() else -1` 자동 감지 패턴 사용

> 첫 번째 실행에서 모델을 다운로드하느라 오래 걸립니다. 다운로드가 완료되면 `~/.cache/huggingface/` 폴더에 캐시되어 다음부터는 빠릅니다.

실습 후 추가 도전 과제 (빠르게 끝난 학생용)

```
# 도전 1: 자기만의 텍스트로 감성 분석 해보기
# → 인스타그램 리뷰, 영화 리뷰, 식당 리뷰 등

# 도전 2: 제로샷 분류에서 카테고리 바꿔보기
# → ["food", "travel", "study", "game"] 등 영어로 지정 (정확도 높음)
# → 심화: ["음식", "여행", "공부", "게임"] 한국어로도 해보고 결과를 비교해보세요!

# 도전 3: 한국어 요약 모델 사용해보기
# summarizer = pipeline("summarization", model="digit82/kobart-summarization",
device=device)
```

2교시 정리

오늘 배운 핵심 개념

개념	핵심 정리	왜 중요한가?
모델 지정	`pipeline(task, model="모델명")`으로 특정 모델 사용	재현성, 최적 모델 선택
한국어 모델	한국어 전용 또는 다국어 모델 선택	한국어 프로젝트 필수
제로샷 분류	학습 없이 자유로운 카테고리로 분류	빠른 프로토타이핑에 필수
번역/요약	태스크별 전용 모델 사용	다양한 NLP 기능 활용
Pipeline 내부	Tokenizer → Model → 후처리 3단계	모델 동작 원리 이해

2교시에서 사용한 핵심 코드 패턴

```
# 패턴 1: 모델 지정하여 Pipeline 사용
classifier = pipeline("sentiment-analysis", model="모델명")

# 패턴 2: 한국어 모델 사용
ko_clf = pipeline("sentiment-analysis", model="nlptown/bert-base-multilingual-uncased-sentiment")

# 패턴 3: 제로샷 분류
zero_clf = pipeline("zero-shot-classification")
result = zero_clf("텍스트", candidate_labels=["카테고리1", "카테고리2"])

# 패턴 4: 번역
translator = pipeline("translation", model="Helsinki-NLP/opus-mt-ko-en")
```

3교시 예고

> Pipeline으로 모델을 사용하면 매번 모델 로딩에 5~10초가 걸립니다. 3교시에서는 Streamlit의 `@st.cache_resource` 캐싱을 이용해서 모델 로딩을 **0초로 만드는 방법**을 배우고, 지금까지 실습한 모델을 **웹 앱**으로 만듭니다.

```
[현재 문제]
사용자 버튼 클릭 → 모델 로드 (5초) → 분석 (0.1초) → 결과 표시
                   ↑ 매번 기다려야 함 😩

[3교시에서 배울 해결책]
사용자 버튼 클릭 → 캐시된 모델 즉시 사용 (0초) → 분석 (0.1초) → 결과 표시
                  ↑ @st.cache_resource ✨
```